WRENFORD BOROUGH COUNCIL · STREET CARE OFFICE

StreetCare AI

How the system works, how data flows through it, and what each part produces. Architecture, data-flow and process diagrams for the pothole and street defect report review platform.

> **Source.** This document follows `StreetCare_AI.pdf` (the project specification), `config/config.py`, the files under `data/` and the schemas workbook. The repository holds the configuration and the data, and every other module is empty so far. So these diagrams show the system **as specified**, using the exact module, function, table and field names the specification requires.

Contents

1. System architecture
2. The six entry points and what each creates
3. Data sources and where they go
4. Data cleaning pipeline
5. Model training
6. Knowledge base indexing
7. Gathering a report's context (no LLM)
8. Screening logic and the assessment
9. Evidence from documents, and predictions
10. The 10-node LangGraph review workflow
11. Agent-to-agent communication
12. The human gate: opening and deciding a review
13. Status lifecycles
14. REST service and access control
15. Database schema (6 tables)
16. Street care chat
17. Dashboard pages by role
18. Output structures
19. End-to-end data lineage

Diagram colour legend

Input data / files · Deterministic code / tools · ML model · LLM (Gemini) agent call · Human decision / gate · Stored output (DB / files) · User interface

1. System architecture

The system has two halves. In the **offline half**, four scripts are run once, in order. They build everything the live system needs: the database, the cleaned data, three trained models and a vector index of the council's 16 documents. In the **online half**, a FastAPI service loads those things once at start-up and serves a Streamlit dashboard. Every figure the dashboard shows comes from the service. The dashboard holds no business rules.

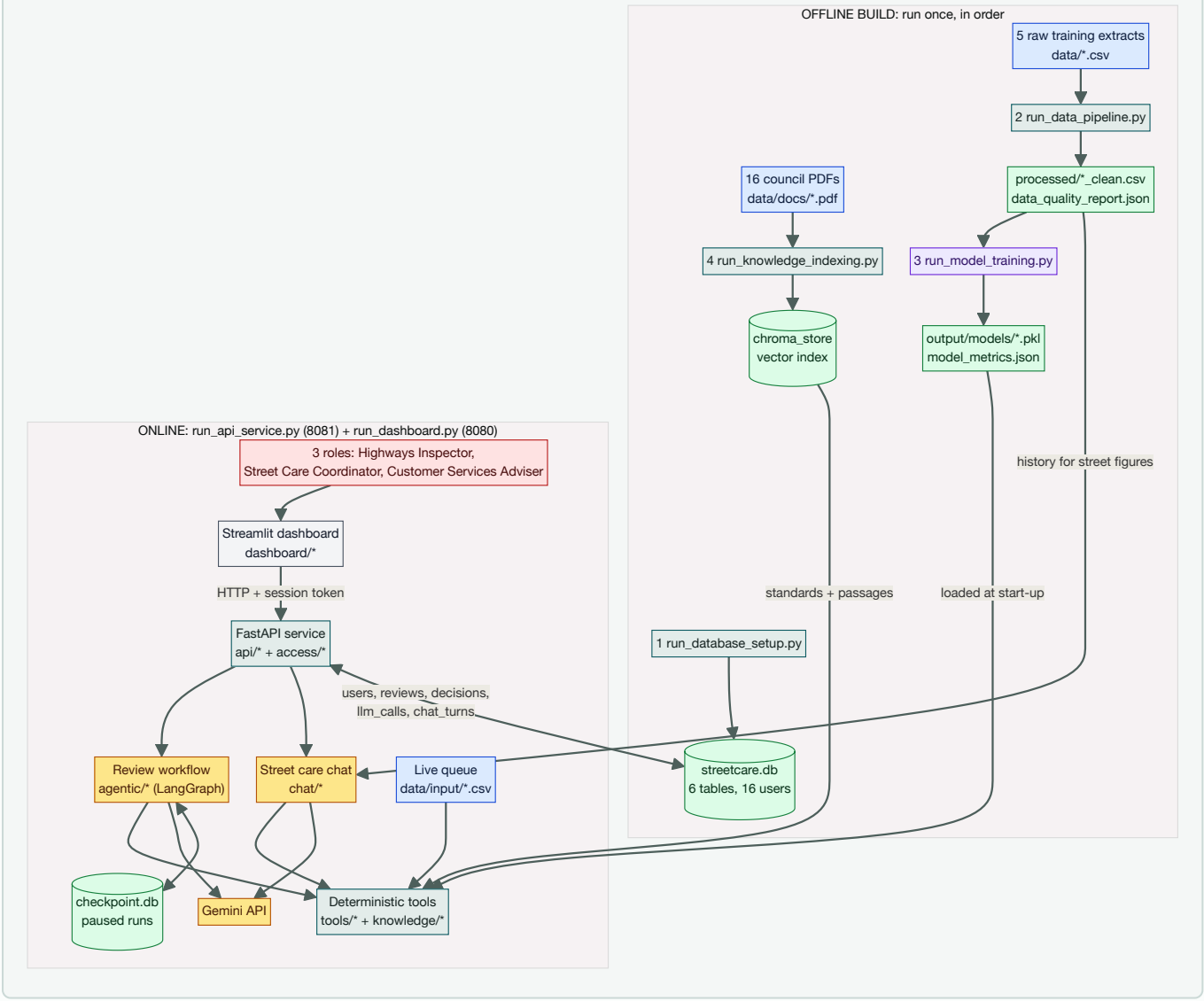


Figure 1. Overall architecture: inputs, the offline build, built artifacts, the online service and the dashboard.

Layers and their packages

Layer	Package	Responsibility
Configuration	`config/config.py`	Every path, spelling mapping, pattern, unit conversion, threshold, role, permission, seed account and message template. It holds **no requirement figure**: those are read from the PDFs.
Data preparation	`data_pipeline/`	Reads raw CSVs as text, makes every value canonical, converts units, keeps the latest revision of each report, profiles data quality.
Modelling	`modeling/`	Repeat-likelihood classifier, crew-hours regressor and sentence-type classifier; scoring and the model registry.
Knowledge	`knowledge/`	PDF text, sections, chunks, the Chroma vector index, and reading each defect standard's requirements.

Tools	`tools/`	Deterministic facts: loading live data, screening, street works, evidence, predictions, retrieval, decision checks, document details.
Agentic	`agentic/`	LangGraph state, the 7 agents (one Gemini call each), the 10 nodes, the graph and its checkpoint.
Persistence	`persistence/`	SQLAlchemy engine, the 6 ORM tables and the repository functions.
Service	`api/`, `access/`	REST routes, Pydantic schemas, password hashing, route permissions and district checks.
Chat	`chat/`	Answers from the documents only; report and street figures answered by code according to the asker's role.
Dashboard	`dashboard/`	Six Streamlit pages that call the service via `api_client.call_service`.

2. The six entry points and what each creates

Every command is run from the `Project/` folder, in this order. Each step reads only what the earlier steps created. The API refuses to start if a saved model is missing.

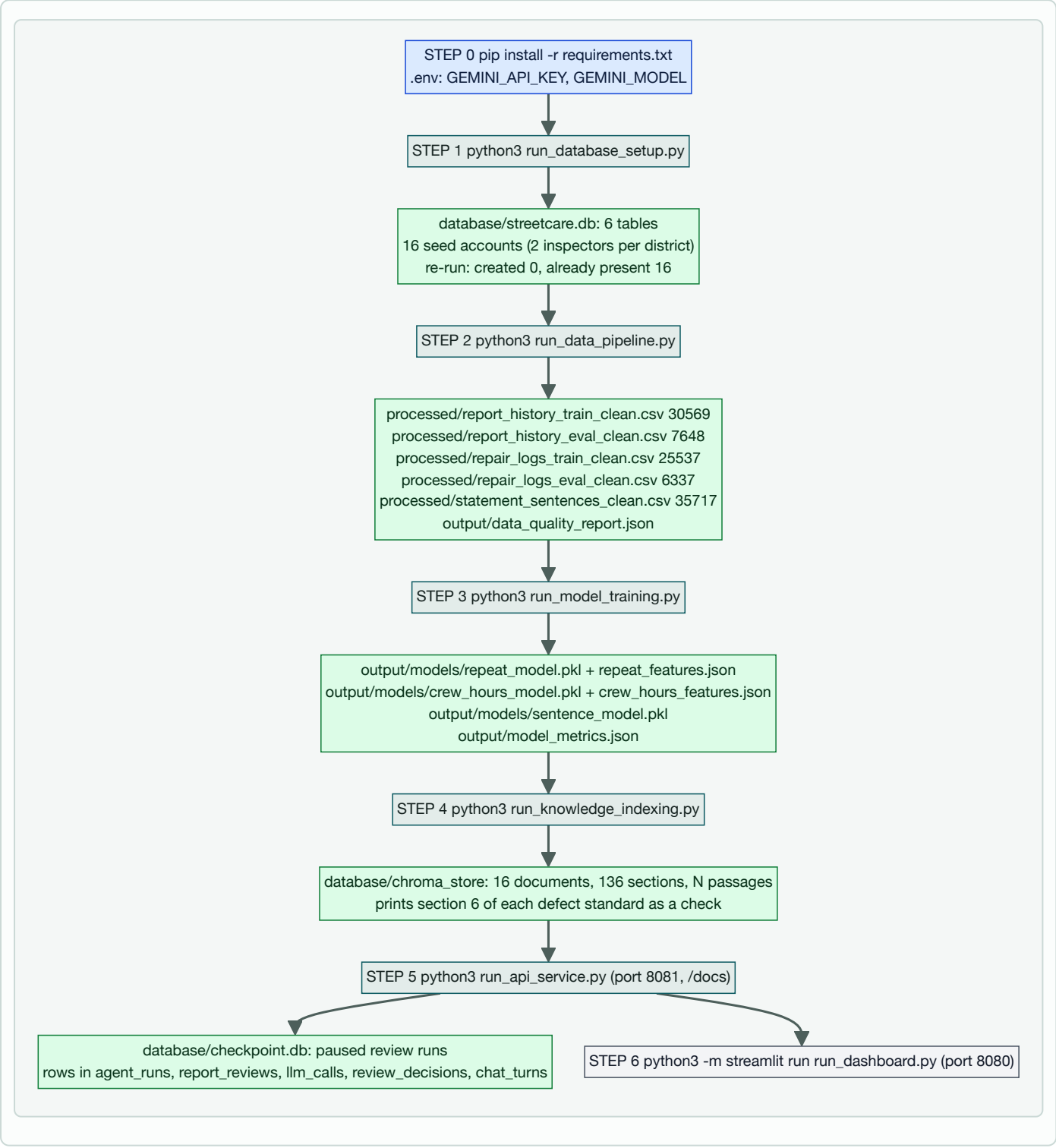


Figure 2. Run order and the files each step leaves behind. Row counts are the specification's expected output.

What each command prints (expected output)

```
ENTRY POINT 2: run_data_pipeline.py
report_history_train            32738 received ->  30569 kept  (2169 removed)
report_history_eval              8202 received ->   7648 kept  (554 removed)
repair_logs_train               26095 received ->  25537 kept  (558 removed)
repair_logs_eval                 6476 received ->   6337 kept  (139 removed)
statement_sentences             36661 received ->  35717 kept  (944 removed)
quality report written to output/data_quality_report.json

ENTRY POINT 3: run_model_training.py
  repeat          held-out ROC AUC N.NNNN  (train N.NNNN, gap +N.NNNN)     minimum 0.80
  crew hours      held-out R2 N.NNNN  MAE N.NNNN hours  (train, gap)       minimum 0.86
  sentence        held-out macro F1 N.NNNN  (train N.NNNN, gap +N.NNNN)    minimum 0.95
A model below its minimum stops the run, naming it, and nothing is saved.
```

3. Data sources and where they go

There are three families of input. **Training extracts** teach the models. **Live files** are the work queue, read on every request. **Documents** are the only source of any requirement, rule or check the system states.

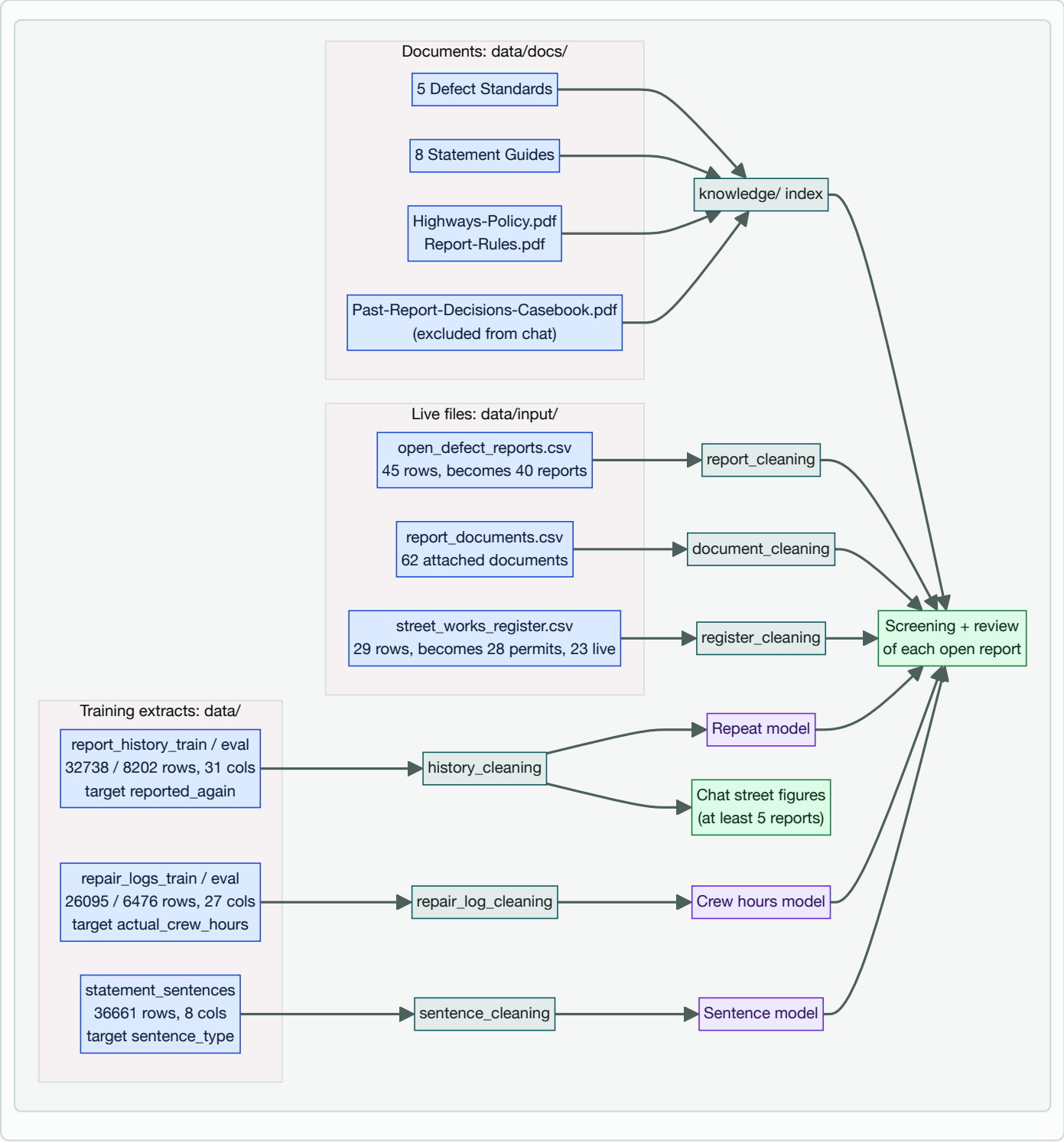


Figure 3. Every input, the cleaning module that reads it, and what consumes the result.

File	Rows → kept	One row is…	Typical defects in the raw data
report_history_train / eval	32738 → 30569 / 8202 → 7648	one revision of a decided report	three reference notations, cm/in/ft/min readings, Y/true/No, lower-case streets, codes like `CW`, earlier revisions, missing target
repair_logs_train / eval	26095 → 25537 / 6476 → 6337	one repaired defect	crew hours in hours, `min`, `hrs`; method, traffic management and completion code in each channel's words; impossible targets

statement_sentences	36661 → 35717	one labelled sentence of an older report's document	HTML markup, `&` entities, no-break spaces, ALL CAPS, fewer than 3 words, missing type
open_defect_reports	45 → 40	a live report revision	same as history; several revisions of one report; no answer column
report_documents	62	a document attached to a live report	markup in `document_text`, mixed reference notations
street_works_register	29 → 28	a street works permit	an exact duplicate (SWP0071240), capitals and padded values

4. Data cleaning pipeline

`run_data_pipeline.py` loops over `DATASET_JOBS`. Each job loads one raw extract as text, cleans it, saves it to `processed/` and profiles it. Every repair lives in `common_cleaning.py`. Each function returns a new table and **refuses** (with a `ValueError` naming the column and values) any notation it does not recognise. It never guesses.

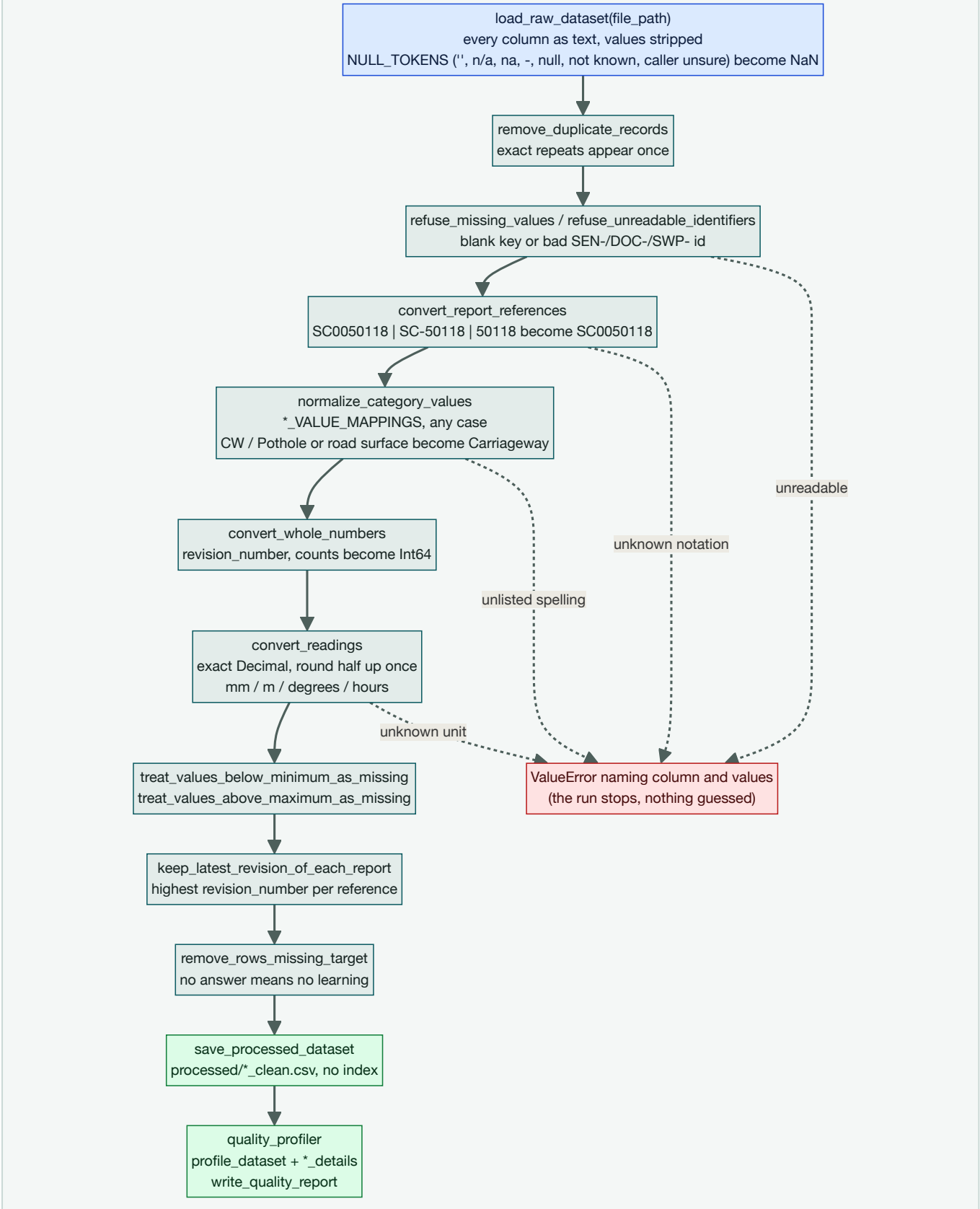


Figure 4. The shared cleaning chain. Each dataset uses the subset of steps its rules need (next figure).

Unit conversions (Report Rules)

Reading	Unit	Conversion	Example
depth, width, water depth	whole mm	cm × 10, in × 25.4	4.5 cm → 45 mm; 1.75 in → 44 mm
reach, clear height, column height	m, 0.1	ft × 0.3048	4 ft → 1.2 m
column lean	degrees, 0.1	none	0–90 kept
crew hours	hours, 0.1	min ÷ 60	45 min → 0.8 h

Text repairs (sentences, documents)

Function	Effect
convert_markup_to_plain_text	strip tags (MARKUP_PATTERN), decode entities, collapse whitespace and no-break spaces
lower_text_written_in_capitals	text with letters but no lower case is lower-cased
sentence_length_characters	recomputed from the cleaned text
MINIMUM_SENTENCE_WORDS = 3	shorter sentences dropped

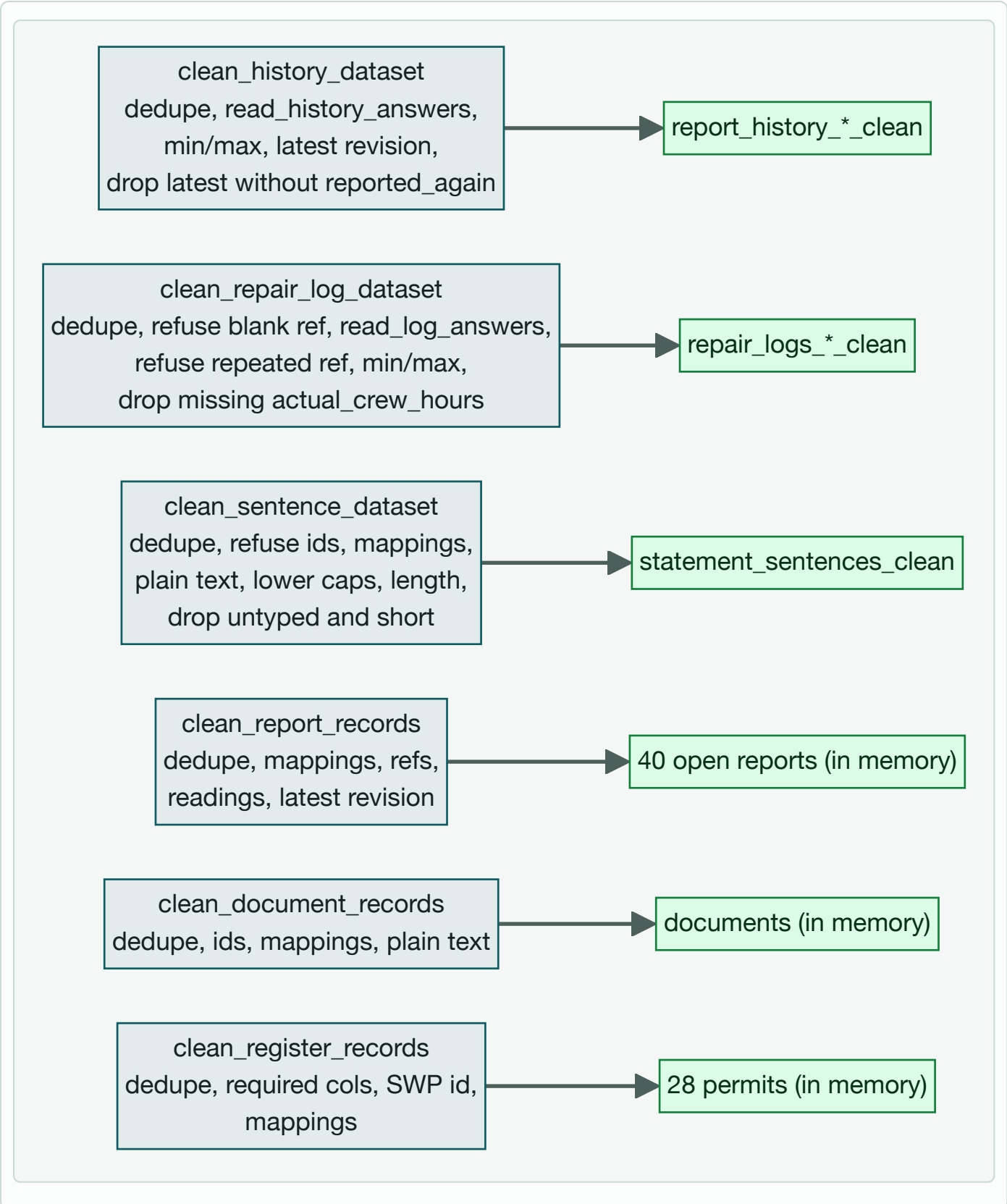


Figure 5. The six dataset-specific cleaning runs. The live three are cleaned on demand by `tools/report_data_tools.py`.

Output structure: `output/data_quality_report.json`

```
{
  "report_history_train": {
    "rows_received": 32738, "rows_kept": 30569, "rows_removed": 2169, "duplicate_rows": ...,
    "rows_missing_target": ..., "answered_earlier_revisions_removed": ..., "columns": 31,
    "missing_cells_as_received": {"defect_depth_mm": ..., ...},
    "missing_cells_after_cleaning": {...},
    "report_references_not_written_canonically": ..., "readings_by_unit_as_written":
{"defect_depth_mm": {"cm": .., "in": ..}},
    "values_below_minimum_as_received": {...}, "values_above_maximum_as_received": {...},
    "category_values_not_written_canonically": {...}, "target_shares": {"No": 0.xxxx, "Yes": 0.xxxx}
  },
  "repair_logs_train": { ..., "rows_with_impossible_target": ..., "median_actual_crew_hours": ... },
  "statement_sentences": { ..., "sentences_with_markup": ..., "sentences_with_encoding_artifacts":
...,
                           "sentences_in_capitals": ..., "rows_with_too_few_words": ...,
"shortest_sentence_words": 3 }
}   // indent 2, keys sorted; the removal reasons add up to rows_removed
```

5. Model training

`run_model_training.py` trains three models on the cleaned train files and scores them on the held-out eval files, which they are never fitted on. Before training, the history gains **requirement counts** (by screening each past report under the standards read from the PDFs) and **trigger flags**. The preparation step (fill gaps, scale numbers, one-hot encode categories) is bundled into the saved pipeline, so a live report with gaps is scored exactly as the model was trained.

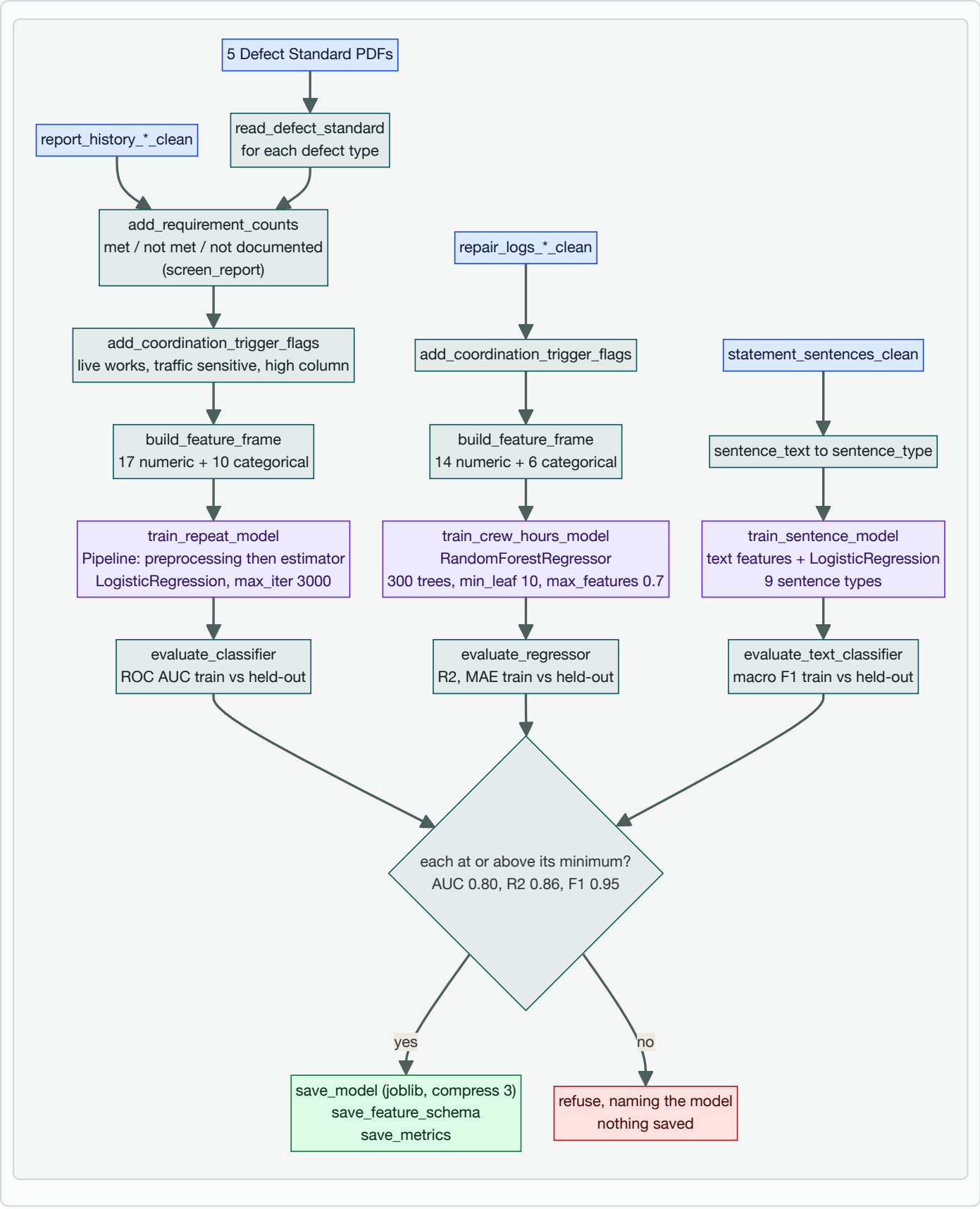


Figure 6. Feature engineering, the three models, held-out evaluation and the minimum-score gate. RANDOM_SEED = 1313 throughout.

Model	Learns from	Target	Inputs	Used live as
Repeat	report history	`reported_again` Yes=1 / No=0	readings, previous reports, planned hours, works counts, 3 requirement counts, 3 trigger flags + defect type, linked type, street category, location fixed, photograph, water/overhang location, highway land, lighting fault, column number	**repeat likelihood**, rounded to 0.01; band HIGH ≥ 0.40, MEDIUM ≥ 0.20, LOW below
Crew hours	repair logs	`actual_crew_hours`	readings, previous reports, works counts, planned hours, 3 trigger flags + 6 categories	**expected crew hours**, rounded to 0.1; overrun if more than 1.3 × planned
Sentence	statement sentences	`sentence_type` (9 types)	sentence text	labels every sentence of a live report's documents

Output structure: `output/model_metrics.json`

```
{ "repeat":     {"held_out_roc_auc": N.NNNN, "train_roc_auc": N.NNNN, "gap": ..., ...},
  "crew_hours": {"held_out_r2": N.NNNN, "held_out_mae": N.NNNN, "train_r2": ..., "gap": ...},
  "sentence":   {"held_out_macro_f1": N.NNNN, "train_macro_f1": ..., "gap": ...} }   // indent 2, keys
sorted
output/models/repeat_features.json        ["defect_depth_mm", "defect_width_mm", ...,
"column_number_given"]
```

> The exact metric key names are defined in the specification's `evaluate_*` sections. The keys above show the shape.

6. Knowledge base indexing

The 16 PDFs are the only source of rules. `run_knowledge_indexing.py` extracts their text, splits each into numbered sections, cuts the sections into overlapping passages and stores them in Chroma with sentence-transformer embeddings. Every passage keeps its document name and section number, so every statement can be traced back to its source. Section reads (for example "section 6 of the Footway standard") are exact lookups by metadata, not similarity searches.

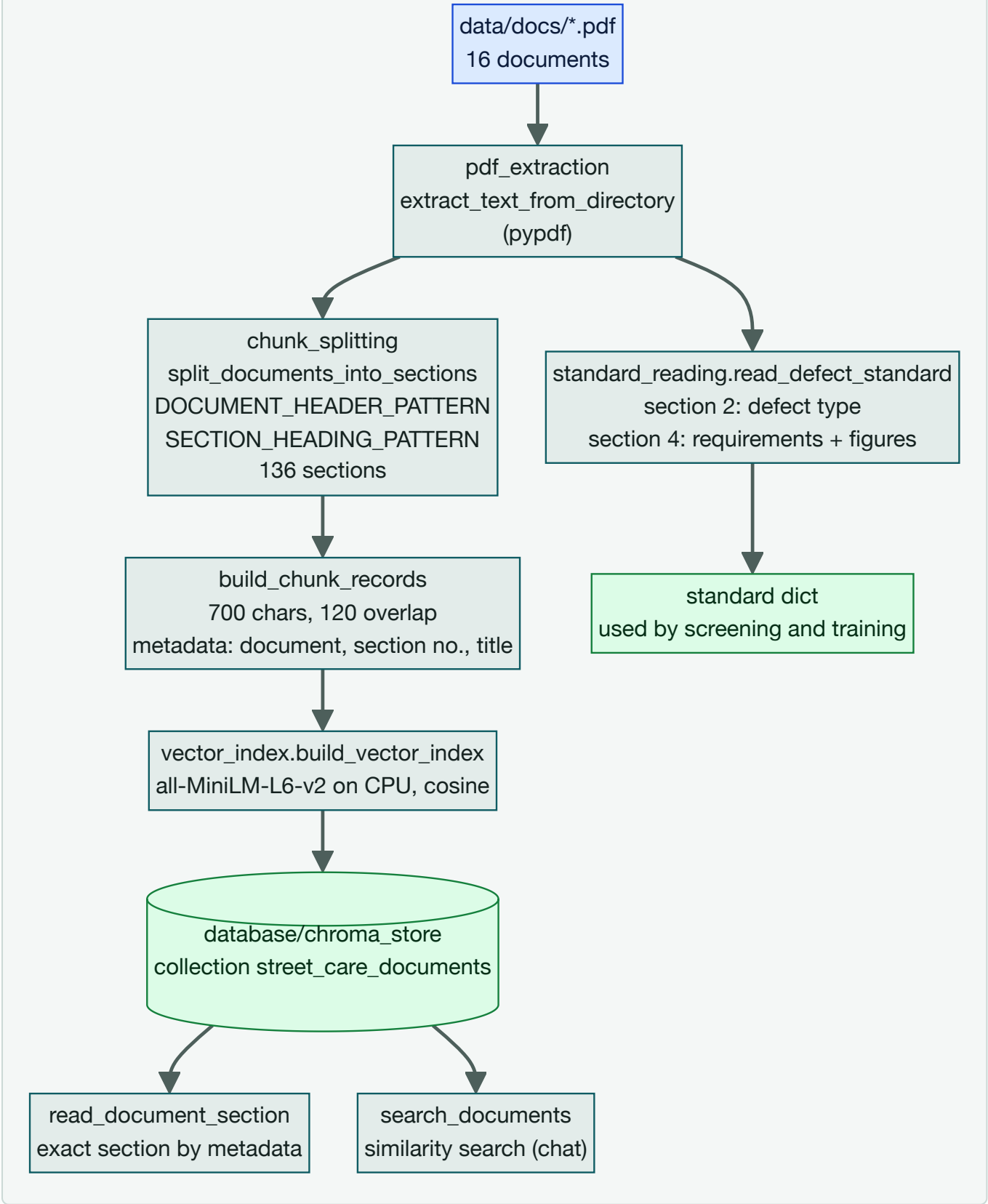


Figure 7. From PDF to searchable, traceable passages, and to a standard's requirements.

Inside a defect standard (shared template)

Section	Content	Read by
2. What this standard covers	"The defect type is Carriageway."	DEFECT_TYPE_PATTERN: checks the document belongs to the type

4. Requirements	numbered, one sentence each, e.g. "3 Depth. … at least 40 millimetres."	REQUIREMENT_SENTENCE_PATTERNS turn each sentence into kind + exact Decimal rules
6. What an inspection records	what the inspector records on site	read_inspection_records (INSPECT documents, indexing check)
7. The works order	what a works order says	read_works_order_guidance
8. The resident update	what a resident update says	read_resident_update_guidance

Output structure: one standard (illustrative, Carriageway)

```
{ "defect_type": "Carriageway", "document_name": "Defect-Standard-Carriageway.pdf",
  "requirements": [
    {"number": 1, "name": "Location fixed", "kind": "location_fixed", "sentence": "...", "rules": {}},
    {"number": 2, "name": "Photograph",     "kind": "photograph",     "rules": {}},
    {"number": 3, "name": "Depth",          "kind": "defect_depth",   "rules": {"minimum":
Decimal("40")}},
    {"number": 4, "name": "Width",          "kind": "defect_width",   "rules": {"minimum":
Decimal("150"), "maximum": Decimal("1500")}} ] }
```

> The figures (40, 150, 1500) come from the PDF at run time. The Footway standard words its depth requirement the same way with a different figure, which is why a requirement is only ever read from the report's own standard.

7. Gathering a report's context (no LLM)

`agentic/nodes.py` loads its resources once at import: the live files, the history, the three models and schemas, the PDF text and the vector collection. `gather_report_context(report_reference)` then establishes every fact about one report with tools only. The API's queue uses this same function for every report it ranks, and the review workflow's first node uses it too.

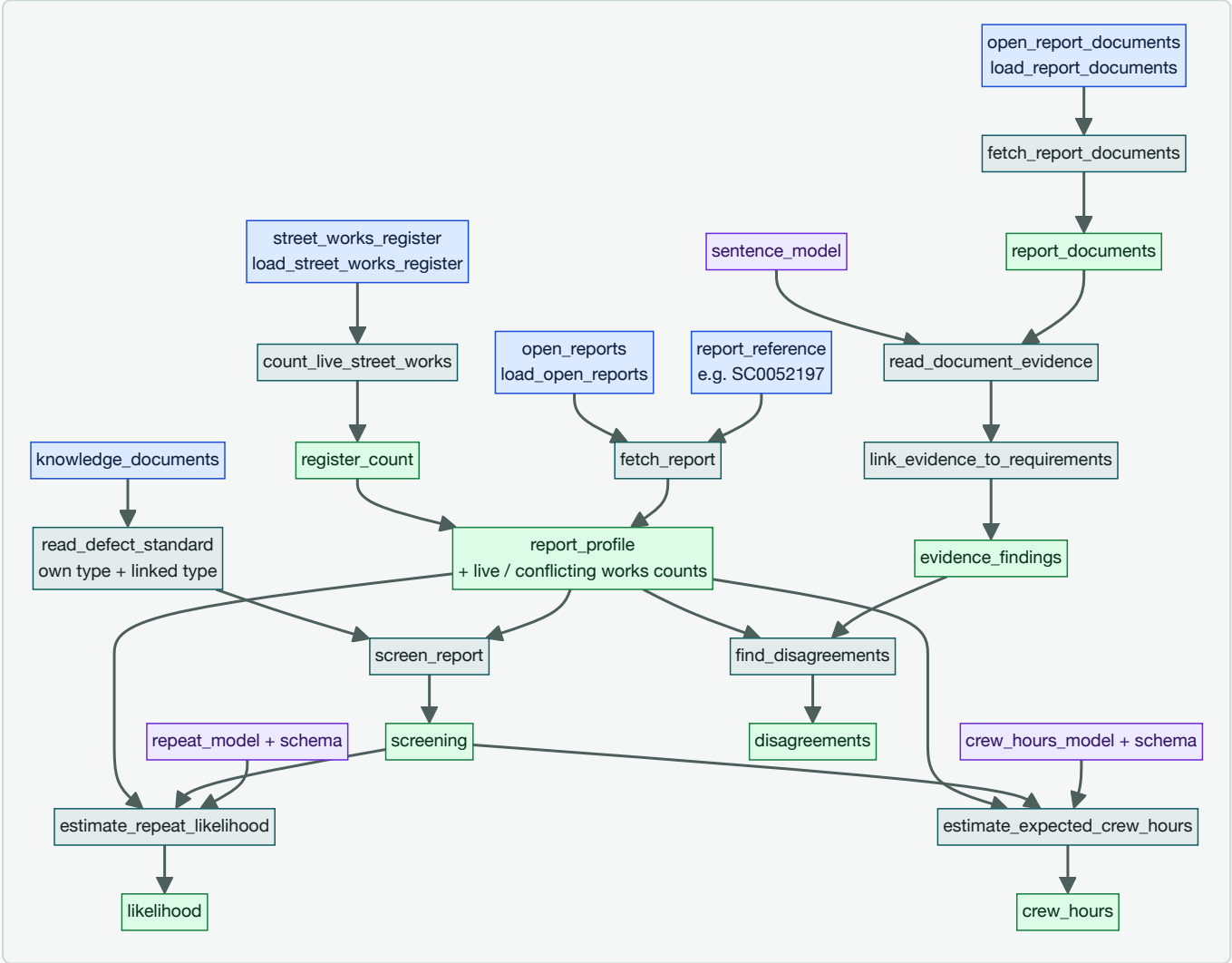


Figure 8. The eight context keys (green), built from live files, the standards and the models. Nothing is written and no LLM is called.

Output structure: `gather_report_context` returns

```
{ "report_profile":    {report_reference, revision_number, street_name, district, defect_type,
linked_defect_type,
                        defect_depth_mm, ..., live_street_works_on_street, conflicting_street_works},
  "report_documents":  [{document_id, document_type, document_title, document_text}, ...],
  "register_count":    {live_street_works_on_street, conflicting_street_works, permits...},
  "screening":         {requirements[...], counts, coordination_triggers, assessment, decisions_open},
  "evidence_findings": [{document_id, sentence_number, sentence_text, sentence_type,
requirement_kinds}, ...],
  "disagreements":     [{answer, statement_figure, report_figure, flag: "DISAGREES WITH FORM" |
"DESCRIBES BLANK ANSWER"}],
  "likelihood":        {repeat_likelihood: 0.37, likelihood_band: "MEDIUM"},
  "crew_hours":        {expected_crew_hours: 3.4, planned_crew_hours: 2.5, overrun_expected: true} }
```

8. Screening logic and the assessment

`screen_report` judges each requirement of the report's own standard, then of its linked type's standard. Each requirement keeps the standard it came from. Separately, `find_coordination_triggers` checks the street. The assessment follows a fixed precedence: **requirements first, then coordination, then order**.

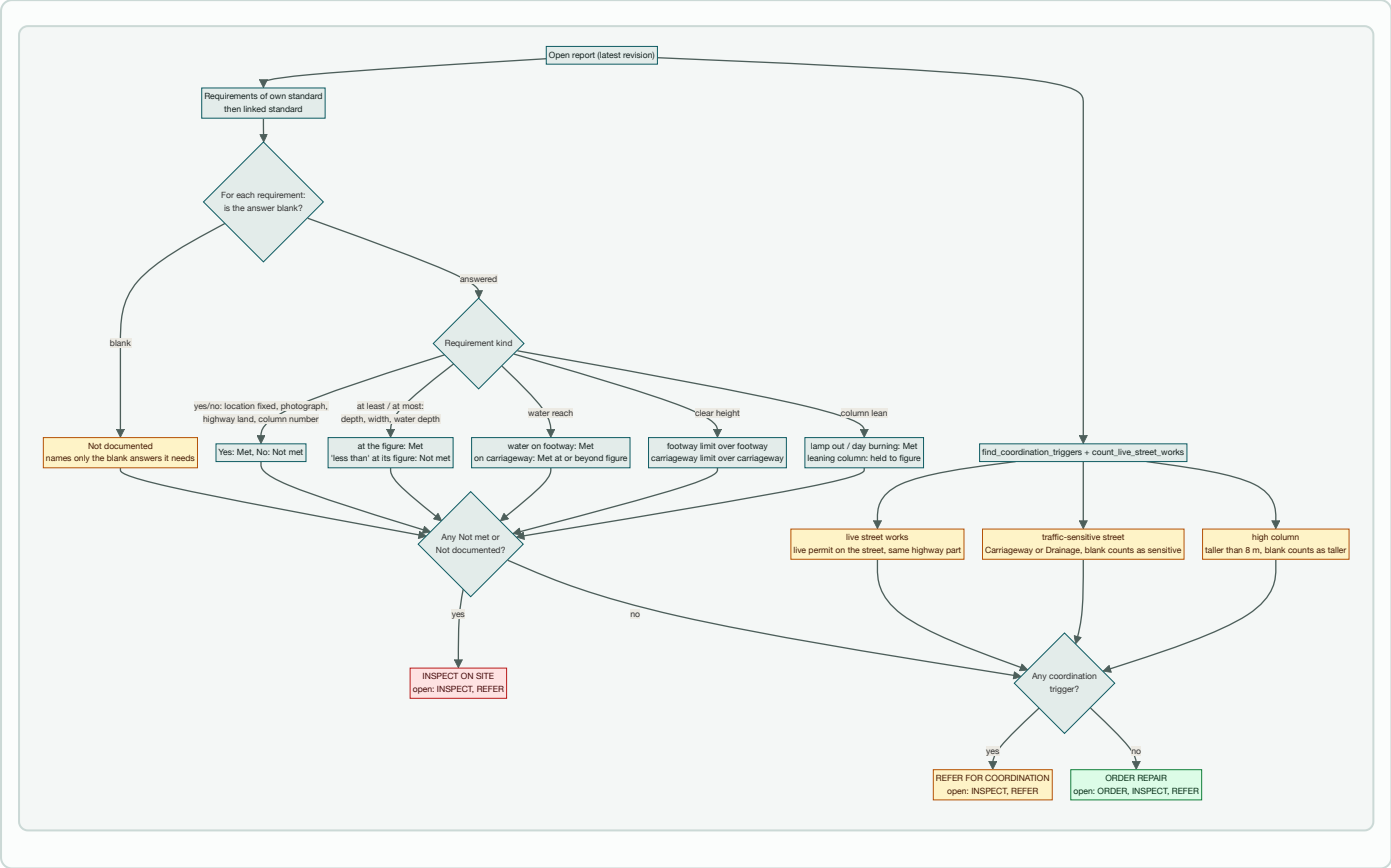


Figure 9. How each requirement is judged, the three coordination triggers, and the assessment with the decisions it leaves open (DECISIONS_OPEN).

> **Worked example (Report Rules §13).** A contact-centre report written `CW` linked `DR` on a street written `TS`: depth 1.25 in → 32 mm (Met); width 3.9 in → 99 mm (Not met, below 100); water 1.5 in → 38 mm (Not met, below 40); water on ROAD, reach "caller unsure" (Not documented, needs the reach only). Met 1, Not met 2, Not documented 1 gives **INSPECT ON SITE**. The traffic-sensitive trigger also applies, but requirements come first.

Output structure: `screening`


```
{ "report_reference": "SC0052163",
  "requirements": [ {"standard": "Carriageway", "number": 3, "name": "Depth", "kind": "defect_depth",
                     "rules": {"minimum": 40}, "result": "Met", "answers_needed": []}, ... ],
  "requirements_met_count": 3, "requirements_not_met_count": 1, "requirements_not_documented_count":
0,
  "coordination_triggers": ["traffic-sensitive street"],
  "assessment": "INSPECT ON SITE",
  "decisions_open": ["INSPECT ON SITE", "REFER FOR COORDINATION"] }
```

9. Evidence from documents, and predictions

Every document attached to a report is read sentence by sentence. The sentence model types each sentence. The type says which requirement kinds the sentence bears on. Only **Measurement** sentences are searched for figures. A figure is compared exactly with the report's own reading and **flagged**, never used to change the answer.

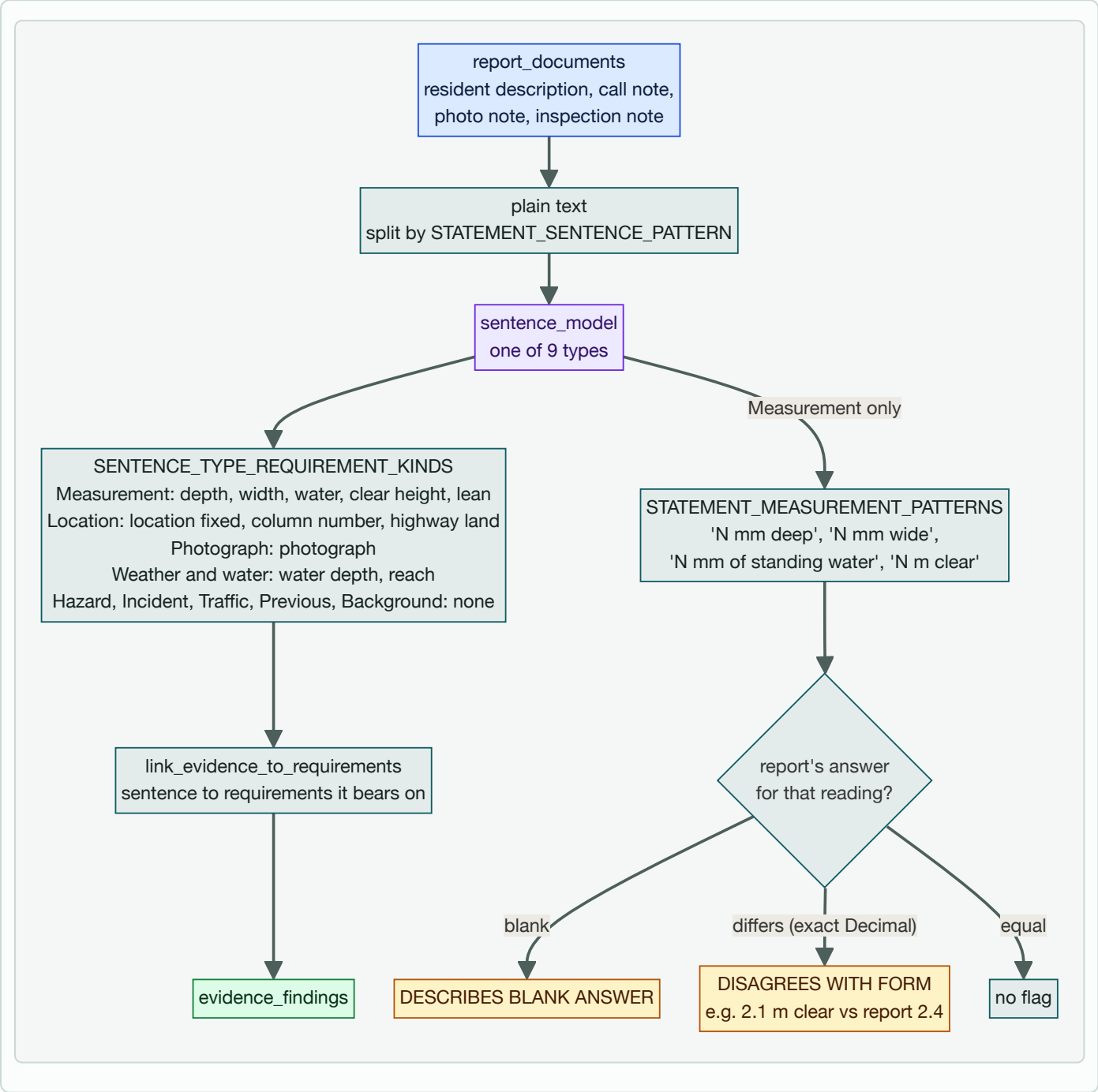


Figure 10a. Reading the documents: typing, linking and flagging.

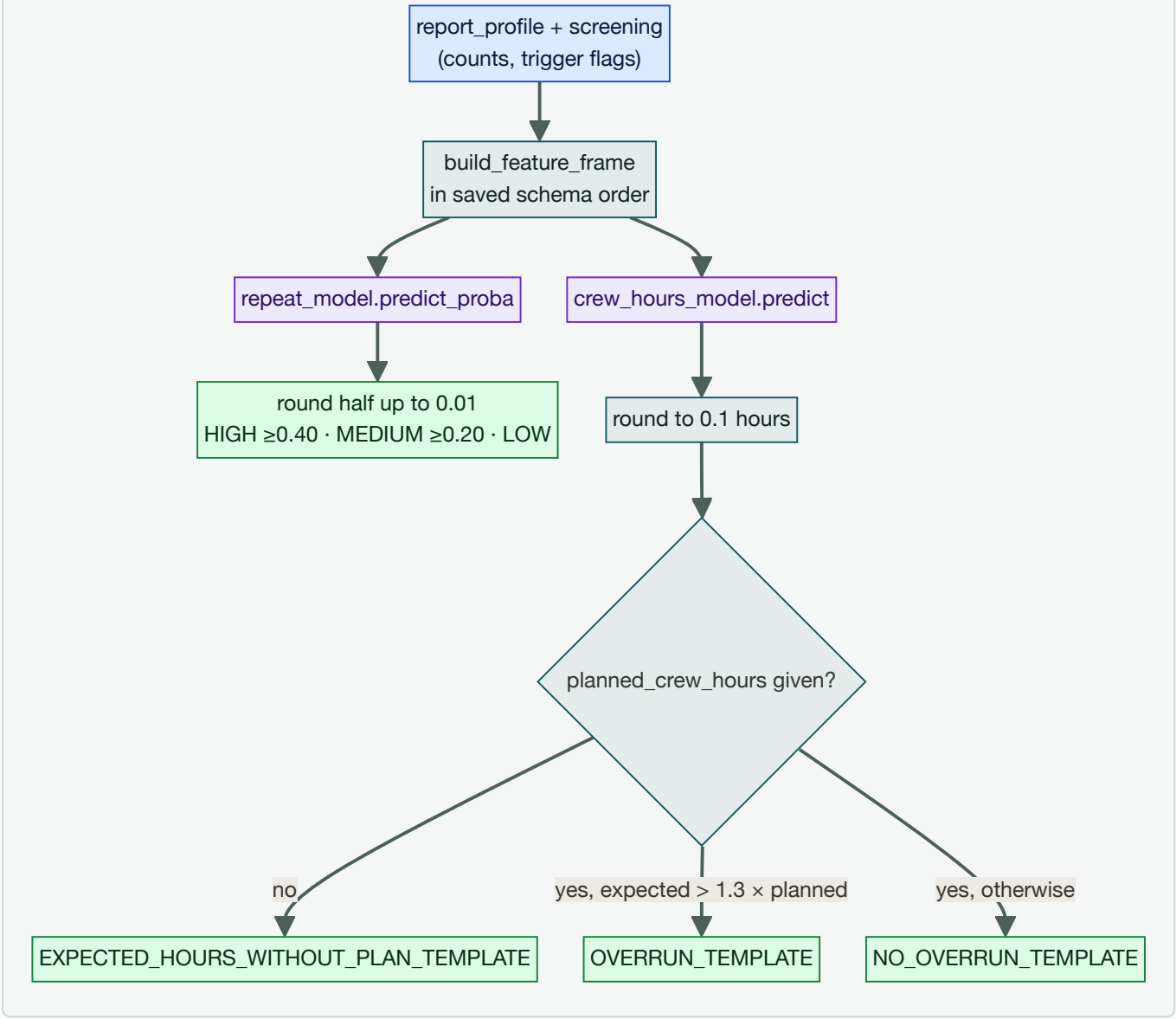


Figure 10b. The two predictions. They inform the inspector, decide nothing, never order the queue, and never appear in a works order or resident update.

10. The 10-node LangGraph review workflow

A review is a LangGraph state machine (`agentic/graph.py → build_workflow`). Five agents interpret the facts before the gate, and up to two more write what the decision calls for. Each agent is **exactly one Gemini call** and calculates nothing. The run pauses at `await_inspector_decision`, and the pause is saved to `checkpoint.db` so it survives a restart. There are exactly two conditional branches, both chosen by `route_report_decision`.

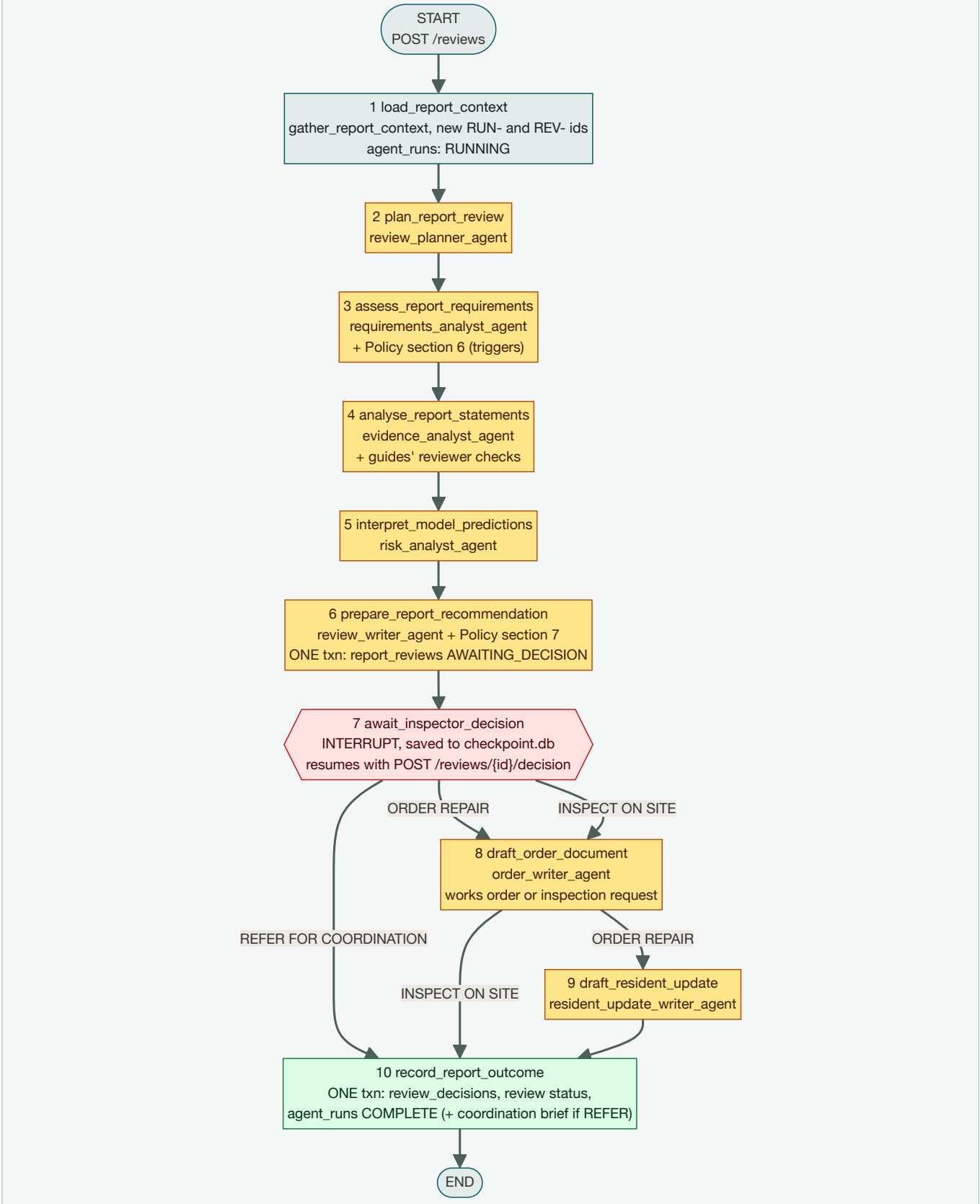


Figure 11. Ten nodes, the human gate and the two conditional branches. Yellow nodes each cost one LLM call.

Decision	Path after the gate	Documents written	LLM calls in total	Final review status
ORDER REPAIR	8 → 9 → 10	works order + resident update	5 + 2 = **7**	REPAIR_ORDERED

INSPECT ON SITE	8 → 10	inspection request	5 + 1 = **6**	INSPECTION_REQUESTED
REFER FOR COORDINATION	10	coordination brief (built by code, no LLM)	**5**	REFERRED_FOR_COORDINATION

State (`ReportReviewState`, a TypedDict)

Keys	Written by
report_reference, coordinator_request, coordinator_id, thread_id	the API, on start
run_id, review_id, report_profile, report_documents, register_count, screening, evidence_findings, disagreements, likelihood, crew_hours	load_report_context
review_plan, requirements_analysis, evidence_review, prediction_analysis	nodes 2–5
review_decision	await_inspector_decision (on resume)
order_document, document_details, resident_update	nodes 8–9
llm_call_count	**added to** by every agent node (a reducer); every other key is replaced

11. Agent-to-agent communication

Agents never talk to each other directly. Each reads what earlier agents wrote **through the state**, plus facts from the tools. Each prompt asks for a fixed set of labelled fields, which `parse_labelled_response` reads back (refusing a reply with missing or extra fields). Every call is logged to `llm_calls` with its tokens and latency.

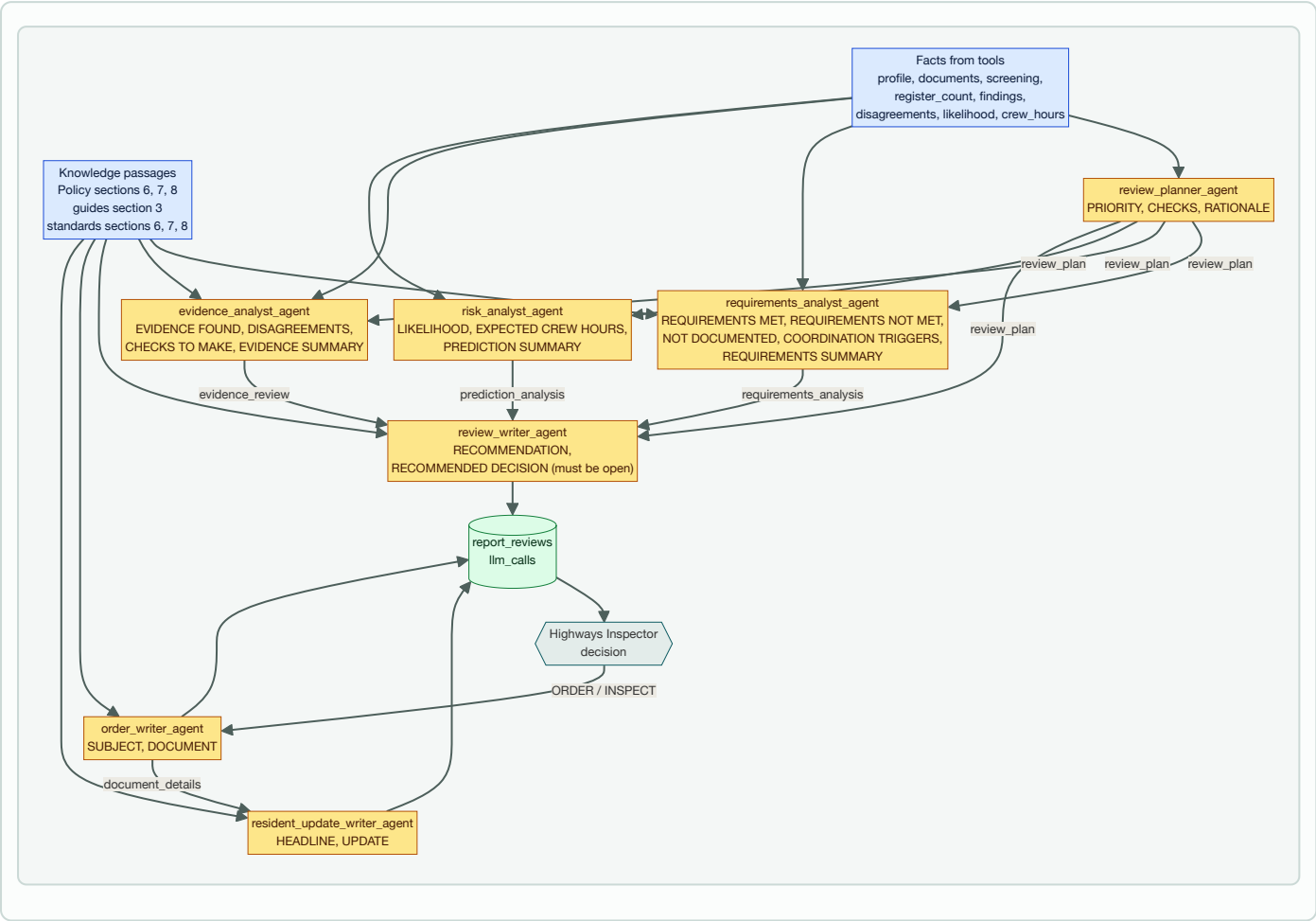


Figure 12. What each agent receives and returns (its labelled fields). Edge labels are state keys.

> **Guard rails on agent output.** Every document is signed "Street Care Office, Wrenford Borough Council". Works orders, inspection requests and resident updates must not mention *likelihood*, *probability* or *expected crew hours*. They must not contain template slots such as `[ ]`, and the resident update names no staff member. The five review write-ups each hold at least 8 words and no two are the same.

12. The human gate: opening and deciding a review

Opening a review (Coordinator or Inspector) runs the workflow up to the gate and returns. Deciding it (a Highways Inspector only) resumes the paused run, possibly from another process after a restart, and finishes it.

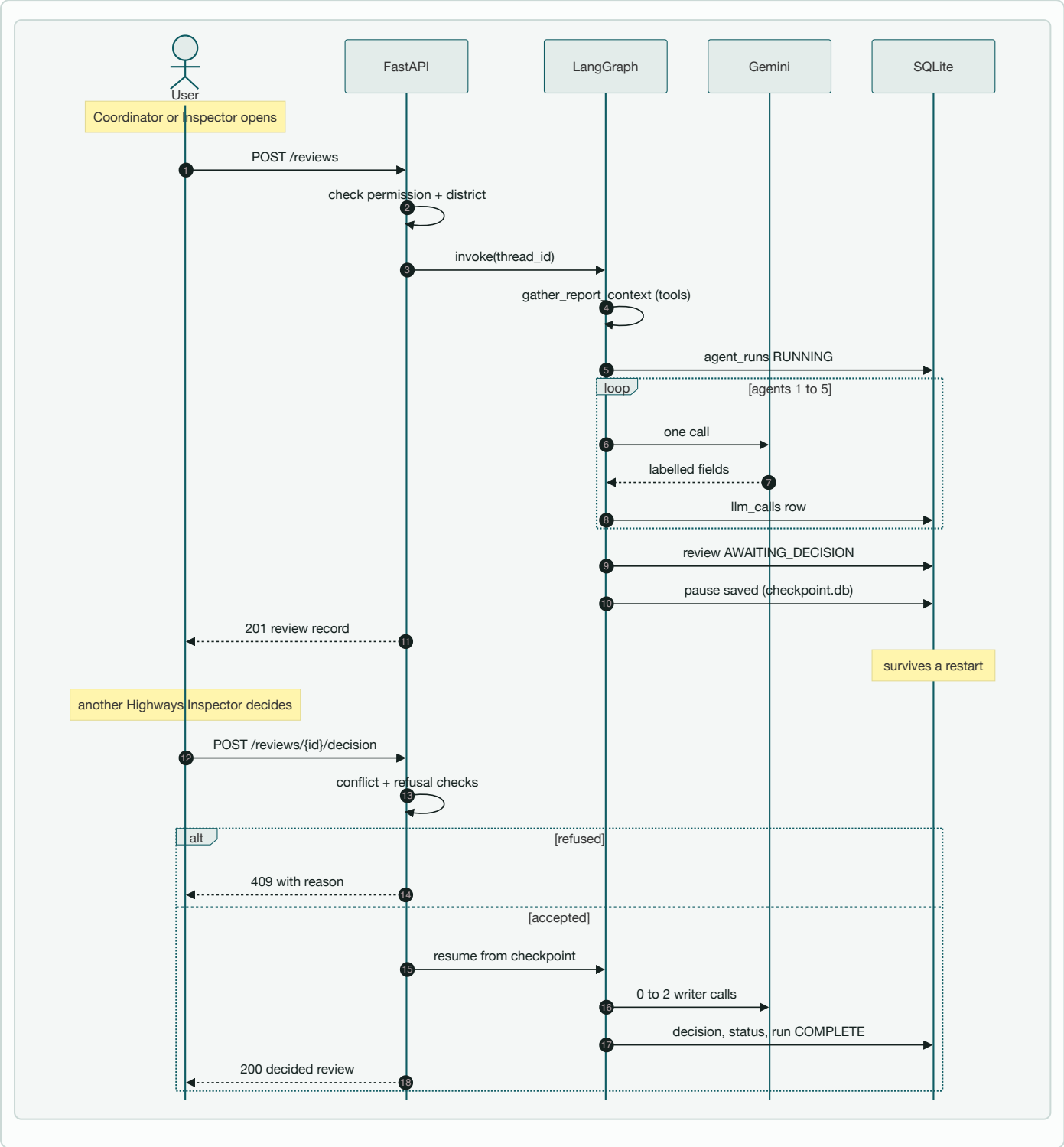


Figure 13. Opening a review, the pause, and deciding it.

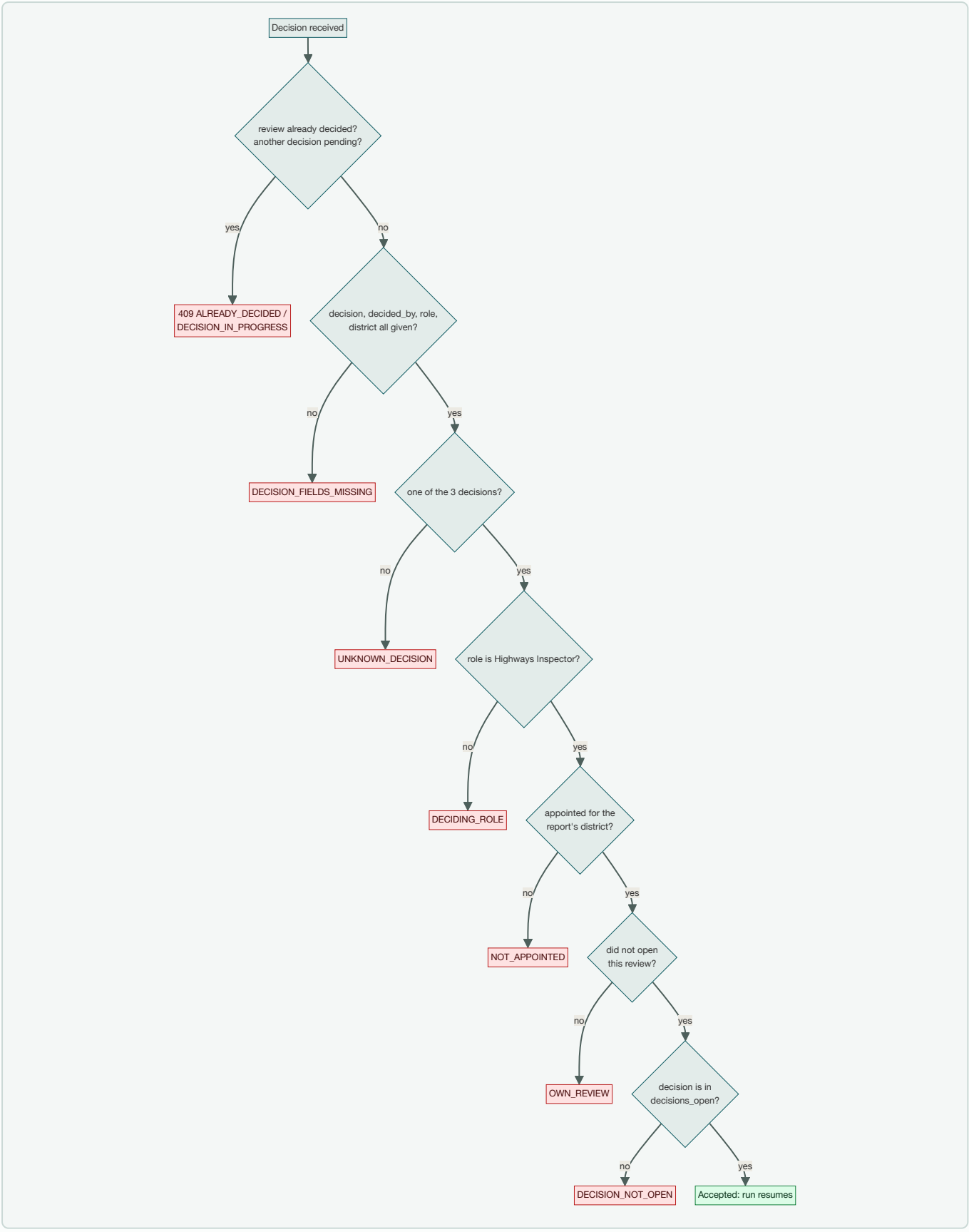


Figure 14. The checks at the gate (find_review_conflict, then find_decision_refusal). The first that applies refuses the decision.

13. Status lifecycles

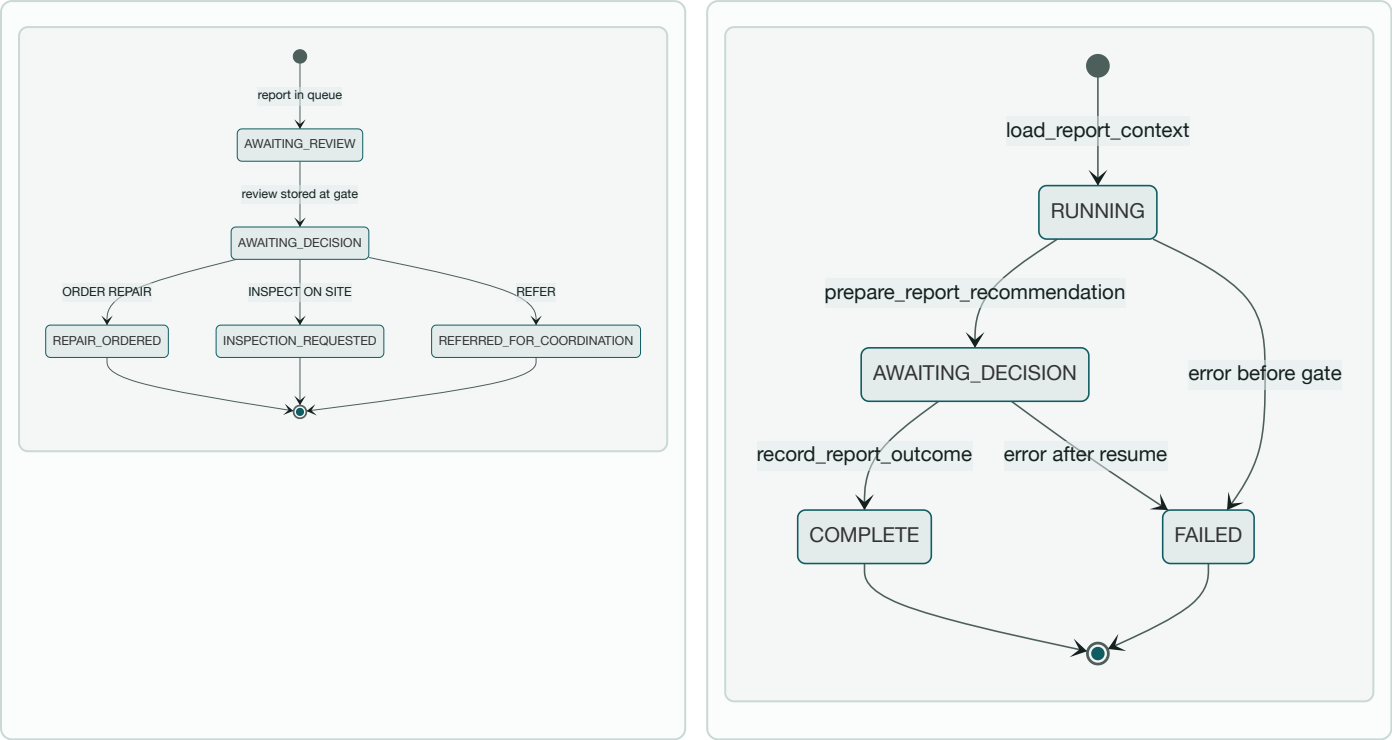


Figure 15. Left: the report and review status shown in the queue. Right: the agent run status in agent_runs.

The report queue

The queue lists every open report, whatever stage its review has reached. Reports whose defect type or linked type is `QUEUE_FIRST_DEFECT_TYPE` come first, then every other report, each group in order of report reference (oldest first). The queue is **never** ordered by a model's figure.

14. REST service and access control

Signing in and out is public. Every other route sits behind `authorize_request`, which resolves the session token to a user and checks the route's permission (`PERMISSION_BY_ROUTE`) against the role (`ROLE_PERMISSIONS`). The service refuses a request on its own, whatever the dashboard shows.

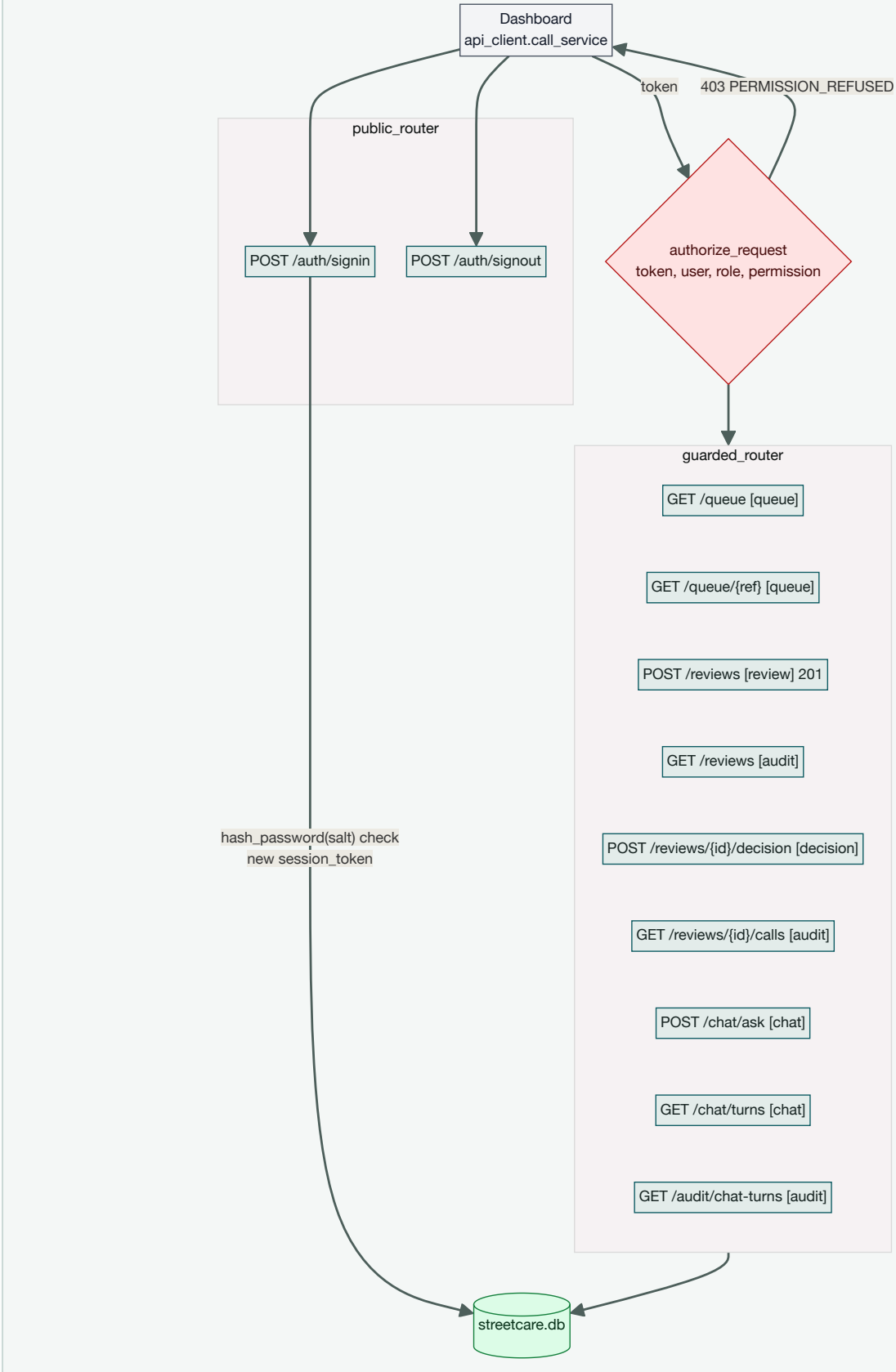


Figure 16. Routes, the permission each needs, and the guard. Only POST /reviews, POST /reviews/{id}/decision and POST /chat/ask can cost LLM calls.

Permission	Highways Inspector	Street Care Coordinator	Customer Services Adviser
queue	✔	✔	–
review (open a review)	✔	✔	–
decision	✔ (own district, not own review)	–	–
audit	✔	✔	–
chat	✔ (full review records)	✔ (full review records)	✔ (status only, never predictions)

Input is validated by Pydantic before any work (**422** on failure). Replies carry only plain Python values. 16 seed accounts: two Highways Inspectors per district (Town Centre, North, South, Villages), plus coordinators and advisers.

15. Database schema (6 tables)

`database/streetcare.db` (SQLite via SQLAlchemy) is the audit record. It holds every run, review, decision, accepted LLM call (with tokens and latency) and chat question. The LangGraph checkpoint is kept separately in `database/checkpoint.db`.

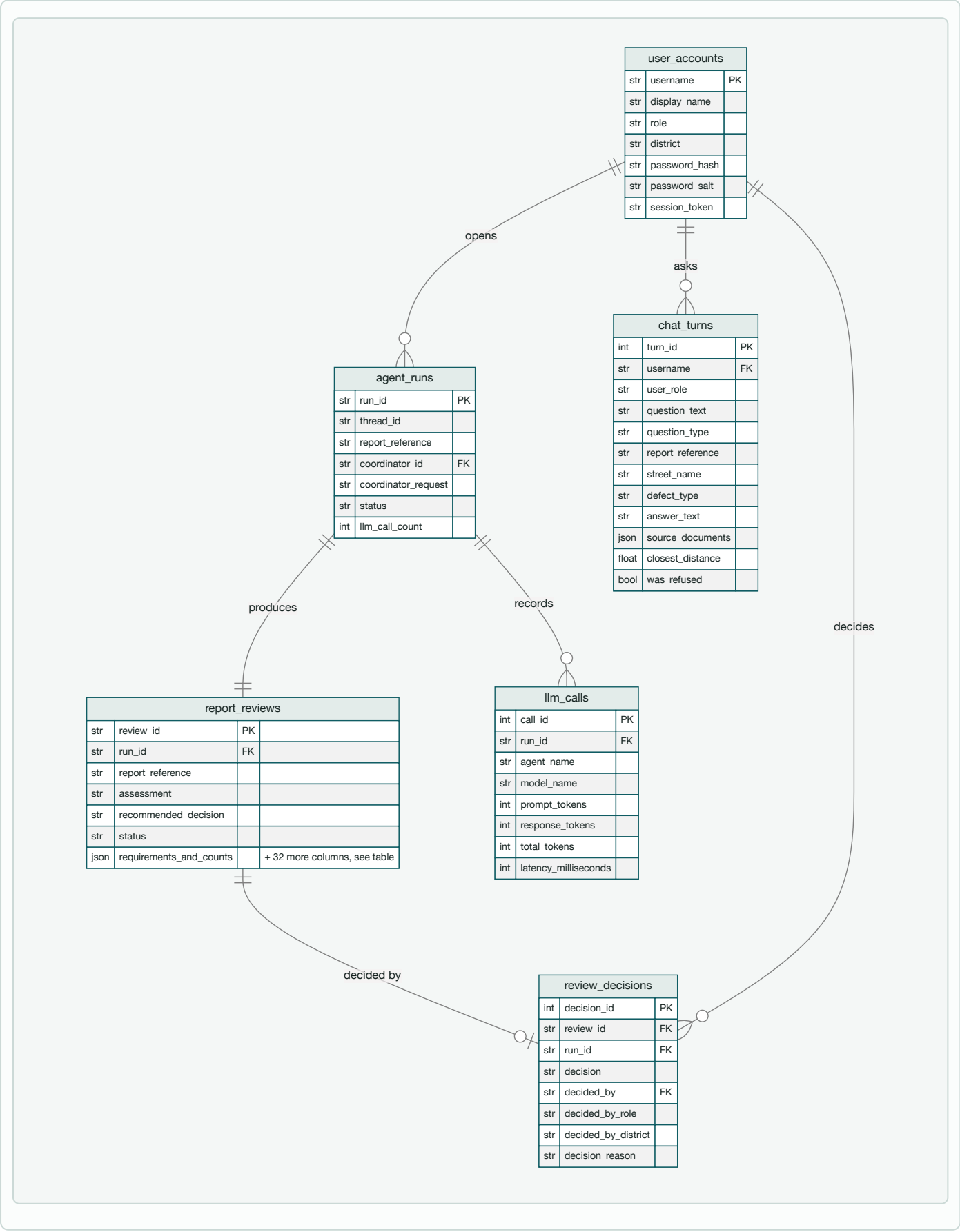


Figure 17. The six tables and their relationships. Column names are exact. JSON-like columns hold serialised lists and dicts; exact storage types follow persistence/models.py.

All 37 columns of `report_reviews`, grouped

Group	Columns
Identity	review_id, run_id, report_reference, revision_number, coordinator_request
Report	defect_type, linked_defect_type, street_name, district
Screening	requirements, requirements_met_count, requirements_not_met_count, requirements_not_documented_count, live_permits, conflicting_permits, coordination_triggers, assessment, decisions_open
Evidence	evidence, disagreements
Predictions	repeat_likelihood, likelihood_band, expected_crew_hours, planned_crew_hours, overrun_expected, crew_hours_text
Agent write-ups	review_plan_text, requirements_analysis_text, evidence_review_text, prediction_analysis_text, recommendation_text, recommended_decision
Outcome	status, order_document_text, document_details, resident_update_text, coordination_brief

16. Street care chat

The chat answers from Wrenford's documents and nothing else (excluding the casebook). One LLM call classifies the question into one of seven types and drafts a document answer. **Code, never the model**, writes every answer about a report or a street, according to the asker's role.

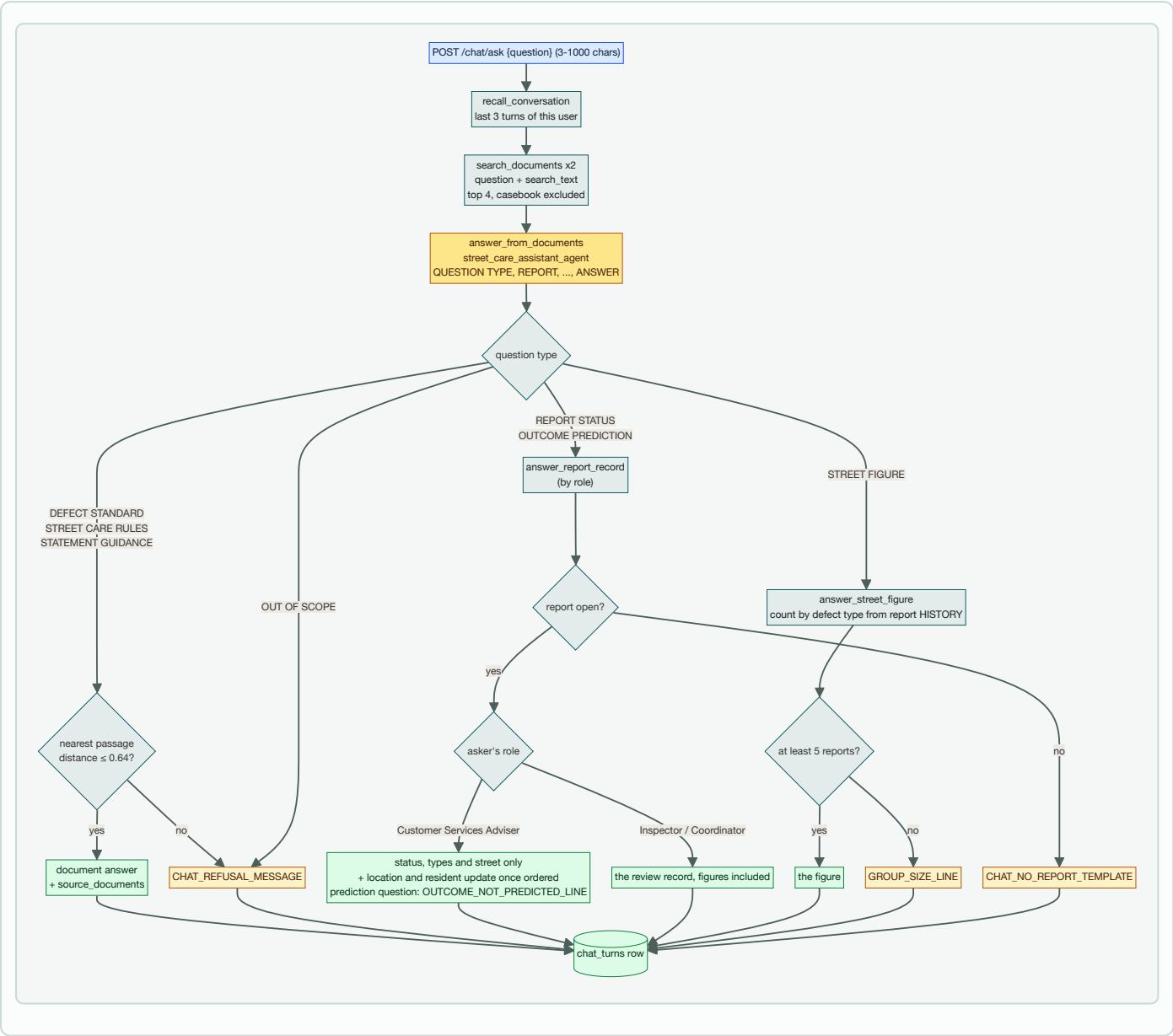


Figure 18. How a chat question is routed and answered, and the privacy rules applied by code.

Output structure: `ChatResponse`

```
{ "question_type": "DEFECT STANDARD", "report_reference": null, "street_name": null, "defect_type":
"Footway",
  "answer": "...", "source_documents": ["Defect-Standard-Footway.pdf"], "was_refused": false }
```

17. Dashboard pages by role

Streamlit (`run_dashboard.py`, port 8080) signs the user in through the service, then shows only the pages in `ROLE_PAGE_ACCESS`. Each page calls the REST service and holds no business rule.

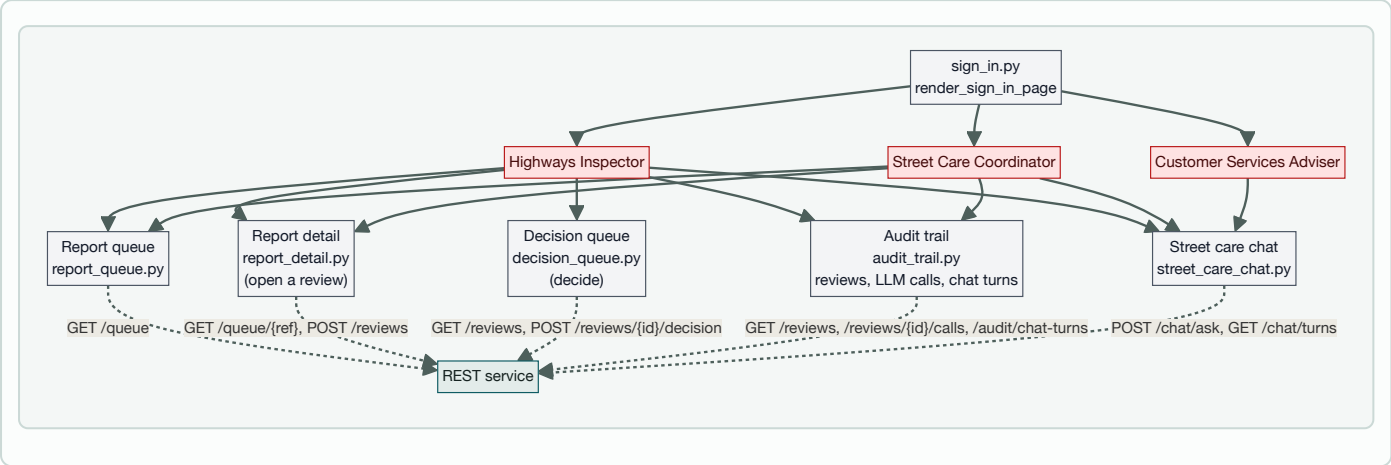


Figure 19. Which role sees which page, and the service calls behind each page.

18. Output structures

Queue row (`GET /queue`, one per open report)


```
{ "queue_position": 1, "report_reference": "SC0052128", "revision_number": 1, "defect_type":
"Vegetation",
  "linked_defect_type": "No linked defect", "street_name": "Long Lane", "district": "Villages",
"report_channel": "StreetCare app",
  "report_status": "AWAITING_REVIEW", "document_count": 2,
  "requirements_met_count": 3, "requirements_not_met_count": 0, "requirements_not_documented_count":
1,
  "live_permit_count": 0, "conflicting_permit_count": 0, "coordination_trigger_count": 0,
"assessment": "INSPECT ON SITE",
  "repeat_likelihood": 0.41, "likelihood_band": "HIGH", "expected_crew_hours": 1.6,
"overrun_expected": false,
  "disagreement_count": 1 }
```


Review calls (`GET /reviews/{id}/calls`)


```
{ "review_id": "REV-3F9A1C0B7E2D", "run_id": "RUN-8C1D2E3F4A5B", "call_count": 7,
"expected_call_count": 7,
  "total_tokens": 18342, "total_latency_milliseconds": 9120,
  "calls": [ {"agent_name": "review_planner_agent", "model_name": "...", "prompt_tokens": ...,
"response_tokens": ...,
              "total_tokens": ..., "latency_milliseconds": ...}, ... ] }
```


Documents each decision writes

Decision	Document	Built from	Written by
ORDER REPAIR	Works order (SUBJECT, DOCUMENT)	build_works_order_details + each standard's section 7	order_writer_agent
	Resident update (HEADLINE, UPDATE)	works order details + each standard's section 8	resident_update_writer_agent
INSPECT ON SITE	Inspection request	build_inspection_request_details + section 6 records + disagreements + Policy section 8	order_writer_agent
REFER FOR COORDINATION	Coordination brief	build_coordination_brief (code only)	no LLM

All model-written documents are signed "Street Care Office, Wrenford Borough Council" and carry no model figure.

Review record (`POST /reviews` 201 and `GET /reviews`)

44 fields: identifiers (review_id, run_id, thread_id, coordinator_id), report metadata, requirements with results, the three counts, live and conflicting permits, triggers, assessment, decisions open, evidence, disagreements, likelihood and band, expected and planned crew hours, overrun and crew_hours_text, the five agent texts, recommended decision, status, outcome documents, llm_call_count and the decision fields once decided.

19. End-to-end data lineage

The whole journey of one open report, from the raw CSV row to the decision recorded in the database.

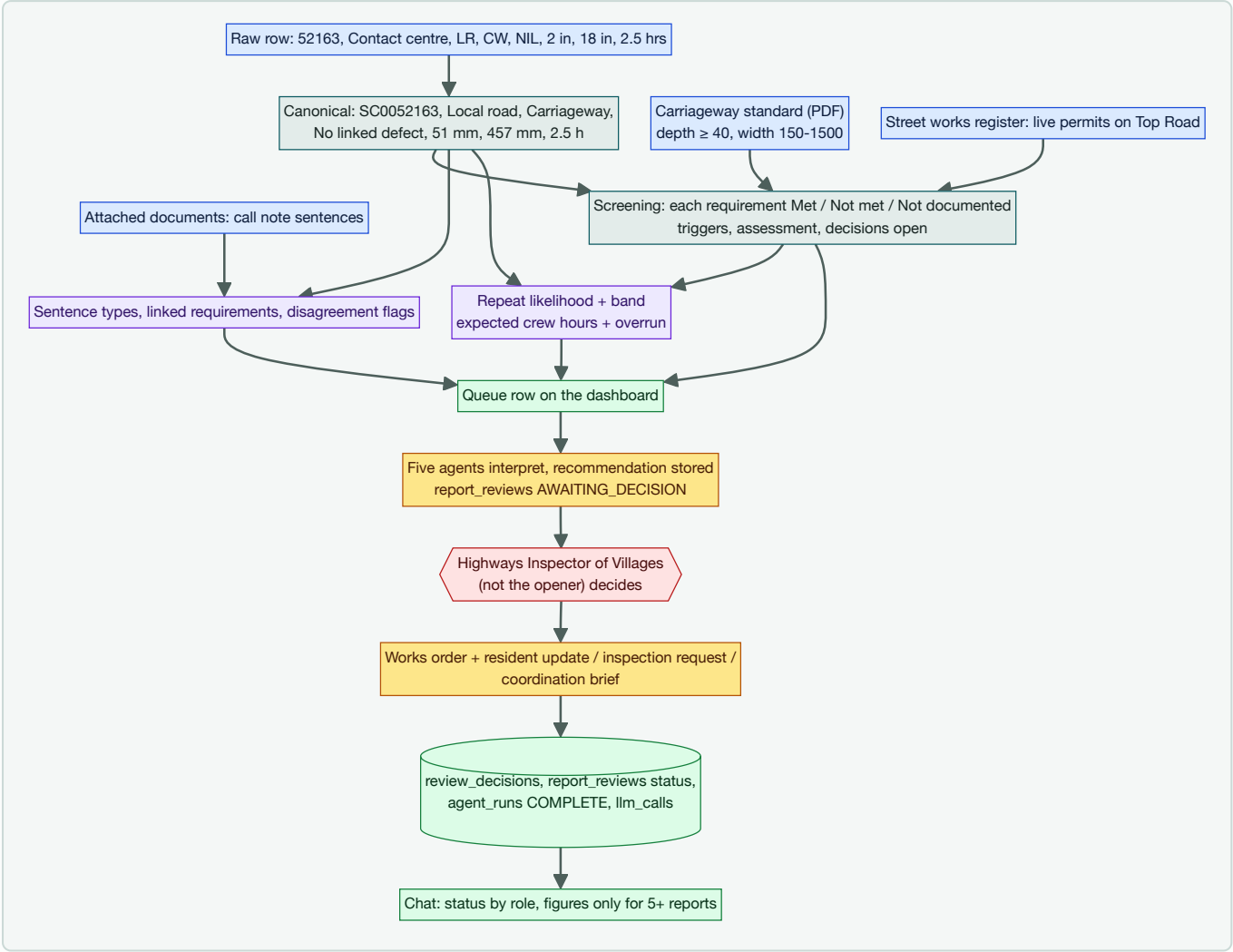


Figure 20. One report's lineage. (Illustrative values; the real result depends on the standards read at run time.)

> **Principles that hold throughout.** (1) No requirement or figure lives in code; it is read from its own standard. (2) Never guess: unknown spellings and units are refused. (3) Agents interpret, tools calculate. (4) Nothing is ordered, written or sent until a Highways Inspector appointed for the district, who did not open the review, decides. (5) Everything is audited: runs, reviews, decisions, LLM calls with tokens and time, and chat turns. (6) Customer services never hears a prediction, and nobody hears a street figure built from fewer than 5 reports.